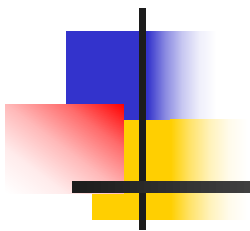


Linux操作系统

授课教师：刘二小

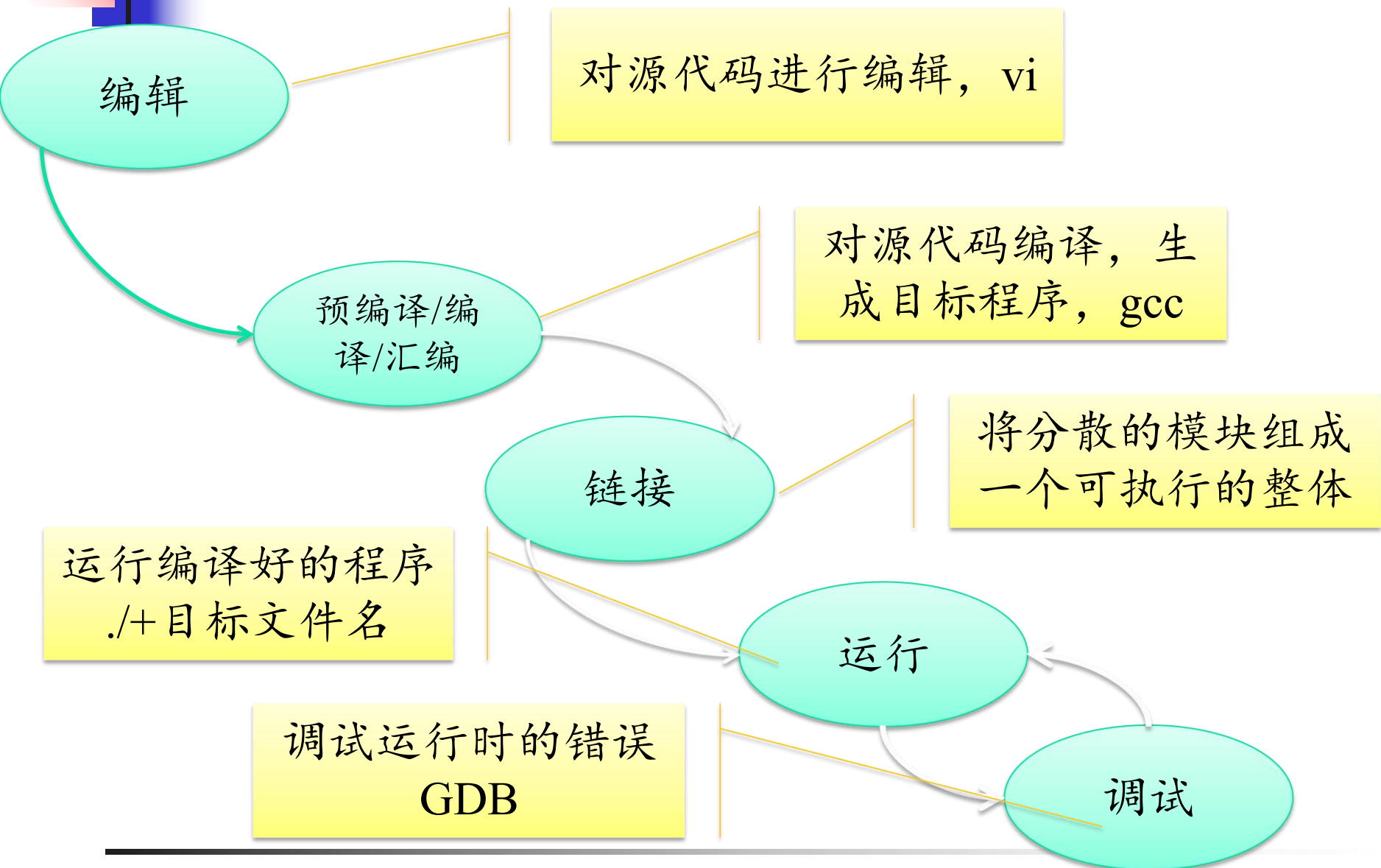
单位：杭州电子科技大学

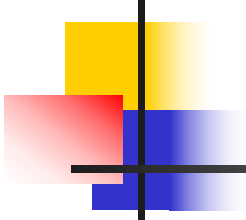
通信工程学院



第九讲 make文件编写

C开发过程





gcc的错误类型及对策

- 第一类：C语法错误

错误资讯：文件source.c中第n行有语法错误(syntax error)。这种类型的错误，一般都是C语言的语法错误，应该仔细检查源代码文件中第n行及该行之前的程序，有时也需要对该文件所包含的头文件进行检查。有些情况下，一个很简单的语法错误，gcc会给出一大堆错误，**我们最主要的是要保持清醒的头脑，不要被其吓倒**，必要的时候再参考一下C语言的基本教材。

- 第二类：头文件错误

错误资讯：找不到头文件head.h(Can not find include file head.h)。这类错误是源代码文件中的包含头文件有问题，可能的原因有头文件名错误、指定的头文件所在目录名错误等，也可能是错误地使用了双引号和尖括号。

Gcc的错误类型及对策

第三类：函数库错误

错误资讯：链接程序找不到所需的函数库，例如：

ld: -lm: No such file or directory

这类错误是与目标文件相链接的函数库有错误，可能的原因是函数库名错误、指定的函数库所在目录名称错误等，检查的方法是使用find命令在可能的目录中寻找相应的函数库名，确定档案库及目录的名称并修改程序中及编译选项中的名称。

第四类：未定义符号

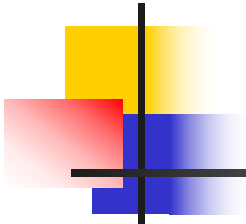
错误资讯：有未定义的符号(Undefined symbol)。这类错误是在链接过程中出现的，可能有两种原因：一是使用者自己定义的函数或者全局变量所在源代码文件，没有被编译、链接，或者干脆还没有定义，这需要使用者根据实际情况修改源程序，给出全局变量或者函数的定义体；二是未定义的符号是一个标准的库函数，在源程序中使用了该库函数，而链接过程中还没有给定相应的函数库的名称，或者是该档案库的目录名称有问题，这时需要使用档案库维护命令ar检查我们需要的库函数到底位于哪一个函数库中，确定之后，修改gcc链接选项中的-l和-L项。

◎ What is Make?

- Make 是用于自动编译、链接程序的实用工具。

◎ Make的优点

- “自动化编译”，一旦写好，只需要一个make命令，整个工程完全自动编译，极大的提高了软件开发的效率。



◎ Make的工作原理

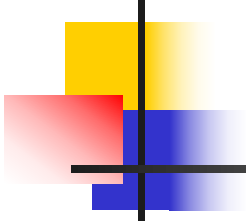
- 当输入make命令之后，会默认的在当前目录下寻找名为“Makefile”或“makefile”的文件。

■ make的使用

- make [options] [target]...

- 常见的选项：

- -f FILE 以指定的FILE 文件作为makefile。
- -n 只打印要执行的命令，但不执行这些命令。
- -s 在执行命令时不显示命令。
- -d 显示调试信息

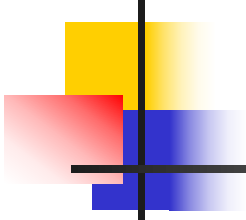


■ **makefile**定义整个工程的编译规则

- 一个工程中的源文件不计其数，其按类型、功能、模块分别放在若干个目录中，makefile定义了一系列的规则来指定，哪些文件需要先编译，哪些文件需要后编译，哪些文件需要重新编译，甚至于进行更复杂的功能操作。

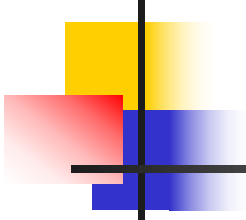
■ 自动化编译

- 只需要一个make命令，整个工程完全自动编译；
- make是一个命令工具，是一个解释makefile中指令的命令工具；一般来说，大多数的IDE都有这个命令，比如：Delphi的make，Visual C++的nmake，Linux下GNU的make。可见，makefile成为了一种在工程方面的编译方法。



Makefile里主要有什么

- 显式规则、隐晦规则、变量定义、文件指示和注释。
 - 显式规则。显式规则说明了如何生成一个或多个目标文件。这是由Makefile的书写者明显指出要生成的文件，文件的依赖文件，生成的命令。
 - 隐晦规则。由于我们的make有自动推导的功能，所以隐晦的规则可以让我们比较粗糙地简略地书写Makefile，这是由make支持的。
 - 变量的定义。在Makefile中要定义一系列的变量，变量一般都是字符串，这个有点类似C语言中的宏，当Makefile被执行时，其中的变量都会被扩展到相应的引用位置上。
 - 文件指示。其包括了三个部分，一个是在一个Makefile中引用另一个Makefile，就像C语言中的include一样；另一个是指根据某些情况指定Makefile中的有效部分，就像C语言中的预编译#if一样；还有就是定义一个多行的命令。有关这一部分的内容，我会在后续的部分中讲述。
 - 注释。Makefile中只有行注释，和UNIX的Shell脚本一样，其注释是用“#”字符。如果你要在你的Makefile中使用“#”字符，可以用反斜框进行转义，如：“\#”。



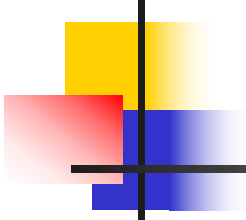
Makefile里的主要规则

一个简单的Makefile文件包含一系列的“规则”，其样式如下：

目标(**target**)：依赖(**prerequisites**)...

<tab>命令(command)

1. target是一个目标文件，可以是Object File，也可以是执行文件。还可以是一个标签(伪目标PHONY，如clean)。
 2. prerequisites是要生成那个target所需要的文件或目标。
 3. command就是make需要执行的命令。(任意的Shell命令)
- 文件的依赖关系，也就是说，target是需要生成的一个或多个文件，它们依赖于 prerequisites中列出的文件，通过执行command来生成target。
 - 一个规则可以有多个命令行，每一条命令占一行。注意：每一个命令行必须以[Tab]字符开始，[Tab]字符告诉 make 此行是一个命令行。make按照命令完成相应的动作。这也是书写 Makefile 中容易产生，而且比较隐蔽的错误。
 - 这就是Makefile的规则，也是**Makefile中最核心的内容**。



makefile文件示例：

- ❖假设：某个正在开发的程序包括prog.c和code.c两个C语言源文件，头文件有prog.h和code.h，且：
- ❖prog.c使用了prog.h和code.h两个头文件中声明的变量；
- ❖最后生成的可执行文件名为test；

则，相应的makefile文件为：

只要文件**test**的时间戳比文件**prog.o**或**code.o**中的任何一个旧，下一行的编译命令将会被执行。

依赖文件位于冒号右边，通常是编译目标文件所需要的其他文件

test: prog.o code.o

目标文件位于冒号左边

gcc -o test prog.o code.o

prog.o: prog.c prog.h code.h

gcc -c prog.c

code.o: code.c code.h

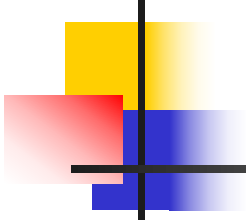
gcc -c code.c

clean:

rm -f *.o

1. 根据依赖文件，生成目标文件，所需执行的命令；
2. 每个命令占一行，且命令行的起始字符必须为**TAB**字符。

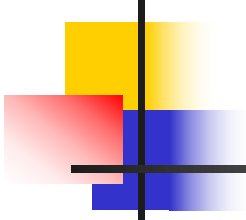
目标文件可以是文件名，或者要执行的动作：**clean**是常用的一种专用目标，用于删除所有的目标模块。



make命令对上述规则的执行顺序:

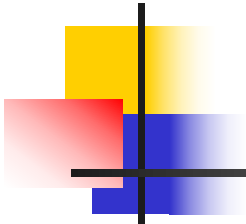
```
test: prog.o code.o
        gcc -o test prog.o code.o
prog.o: prog.c prog.h code.h
        gcc -c prog.c
code.o: code.c code.h
        gcc -c code.c
clean:
        rm -f *.o
```

1. 在检查文件prog.o和code.o的时间戳之前，make会在下面的行中寻找以prog.o和code.o为目标的规则；
2. 在第三行中找到了关于prog.o的规则，该文件的依赖文件是prog.c、prog.h和code.h；
3. 同样，make会在后面的规则行中继续查找这些依赖文件的规则，
4. 如果找不到，则开始检查这些依赖文件的时间戳，如果这些文件中任何一个的时间戳比prog.o的新，make将执行“gcc -c prog.c -o prog.o”命令，更新prog.o文件；



◆ 以同样的方法，接下来对文件code.o做类似的检查，依赖文件是code.c和code.h。当make执行完所有这些嵌套的规则后，make将处理最顶层的test规则。

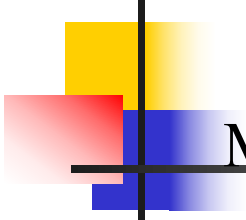
◆ 如果关于prog.o和code.o的两个规则中的任何一个被执行，至少其中一个.o目标文件就会比test新，那么就要执行test规则中的命令，将prog.o和code.o链接成目标文件test。



通过以上的分析过程，可以看到make的优点：

◆ 因为.o文件依赖.c源文件，源码文件里一个简单改变都会造成重新编译，并根据规则依次由下到上执行编译过程，直到最终的可执行文件被重新链接；

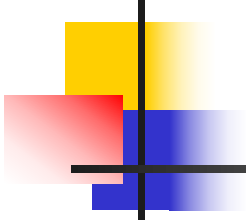
◆ 例如，当改变一个头文件的时候，由于所有的依赖关系都在makefile里记录了，因此，make命令可以自动的重新编译所有那些因依赖这个头文件而改变了的源码文件，如果需要，再进行重新链接。



Make命令的工作过程如下：

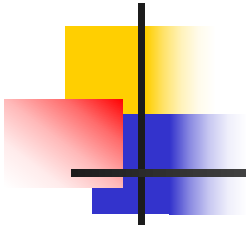
- ① 读入makefile文件；
- ② 初始化文件中的变量；
- ③ 推导隐式规则，并分析所有规则；
- ④ 为所有的目标文件创建依赖关系链；
- ⑤ 根据依赖关系和时间数据，确定哪些目标文件要重新生成；
- ⑥ 执行相应的生成命令。

在默认方式下，输入make命令，就可以调用工作：
在当前目录下寻找名字为makefile的文件，从第一个规则开始执行。



即：

make在寻找到文件之后，make会一层又一层地去找文件的依赖关系，直到最终编译出第一个目标文件。在找寻的过程中，如果出现错误，比如最后被依赖的文件找不到，那么make就会直接退出，并报错，而对于所定义的命令的错误，或是编译不成功，make根本不理。make只管文件的依赖性，即，如果在我找了依赖关系之后，冒号后面的文件还是不在，那么对不起，我就不工作啦。



准备文件file1.c

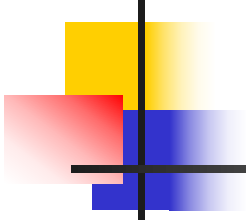
```
#include <stdio.h>
#include "file2.h"
int main()
{
    printf("print file1\n");
    File2Print();
    return 0;
}
```

准备文件file2.c

```
#include "file2.h"
void File2Print()
{
    printf("Print file2\n");
}
```

准备文件file2.h

```
#ifndef FILE2_H_
#define FILE2_H_
#include<stdio.h>
void File2Print();
#endif
```



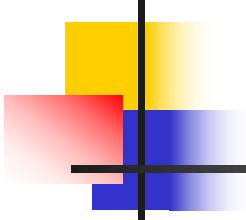
编写Makefile文件

```
helloworld:file1.o file2.o
    gcc file1.o file2.o -o helloworld
```

```
file1.o:file1.c file2.h
    gcc -c file1.c -o file1.o
```

```
file2.o:file2.c file2.h
    gcc -c file2.c -o file2.o
```

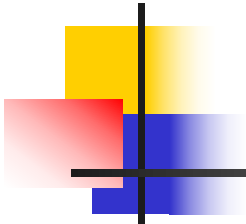
```
clean:
    rm -rf *.o helloworld
```



helloworld : file1.o file2.o

gcc file1.o file2.o -o helloworld

- helloworld依赖file1.o file2.o两个文件编译出helloworld可执行文件。-o表示指定的目标文件名。
- 若file1.o和file2.o都存在，但是其中一个比helloworld新或helloworld不存在时，执行下面的gcc命令。
- 若file1.o和file2.o有任意一个不存在，假设是file1.o，会先找到file1.o的生成关系，生成file1.o后，再执行下面的gcc命令。



file1.o : file1.c file2.h

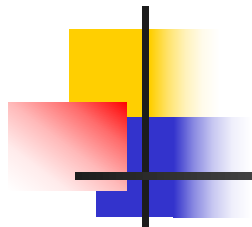
gcc -c file1.c -o file1.o

- file1.o依赖file1.c 和file2.h这两个文件，编译出file1.o文件。
- 若依赖文件中有任何一个文件不存在或比file1.o新，都会重新执行下面的gcc命令。
- -c表示gcc 只把给它的文件编译成.o文件，用源码文件的文件名命名但把其后缀由“.c”变成“.o”。在这句中，可以省略-o file1.o，编译器默认生成file1.o文件，这就是-c的作用。

file2.o : file2.c file2.h

gcc -c file2.c -o file2.o

这两句的功能同上。

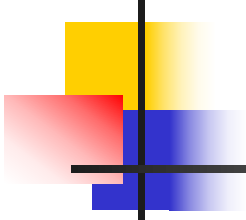


clean:

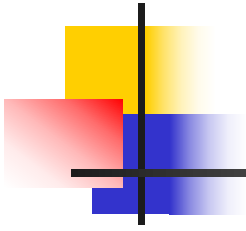
```
rm -rf *.o helloworld
```

#当用户输入make clean命令时，会执行rm指令，其功能是删除*.o 和helloworld文件。

写好Makefile文件，在命令行中直接键入make命令，就会执行Makefile中的内容了

- 
-
- 假设一个工程有3个头文件，和8个源文件，我们按照那三个规则写出的makefile应该是下面的这个样子的。

```
edit :main.o kbd.o command.o display.o \  
insert.o search.o files.o utils.o  
        cc -o edit main.o kbd.o command.o display.o \  
insert.o search.o files.o utils.o  
main.o : main.c defs.h  
        cc -c main.c  
kbd.o : kbd.c defs.h command.h  
        cc -c kbd.c  
command.o : command.c defs.h command.h  
        cc -c command.c
```



display.o : display.c defs.h buffer.h

cc -c display.c

insert.o : insert.c defs.h buffer.h

cc -c insert.c

search.o : search.c defs.h buffer.h

cc -c search.c

files.o : files.c defs.h buffer.h command.h

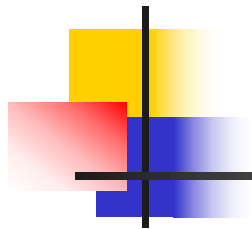
cc -c files.c

utils.o : utils.c defs.h

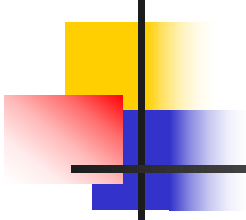
cc -c utils.c

clean :

rm edit main.o kbd.o command.o display.o \
insert.o search.o files.o utils.o



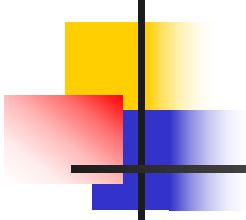
- 在这个makefile中，
- 目标文件（target）包含：执行文件edit和中间目标文件（*.o）。依赖文件（prerequisites）就是冒号后面的那些 .c 文件和 .h文件。
- 每一个 .o 文件都有一组依赖文件，而这些 .o 文件又是执行文件 edit 的依赖文件。
- 依赖关系的实质上就是说明了目标文件是由哪些文件生成的，换言之，目标文件是哪些文件更新的。



make具体工作

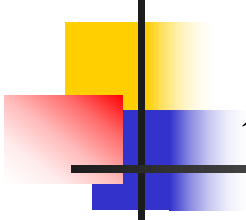
在默认的方式下，也就是我们只输入make命令。那么：

- 1、make会在当前目录下找名字叫“Makefile”或“makefile”的文件。
- 2、默认的情况下，make执行的是Makefile中的第一个规则。此规则的第一个目标称之为“最终目的”或者“终极目标”。在上面的例子中，它会找文件中的第一个目标文件（target），即“edit”这个文件，并把这个文件作为最终的目标文件。
- 3、如果edit文件不存在，或是edit所依赖的后面的.o文件的文件修改时间要比edit这个文件新，那么，他就会执行后面所定义的命令来生成edit这个文件。
- 4、如果edit所依赖的.o文件也不存在，那么make会在当前文件中找目标为该.o文件的依赖规则，如果找到则再根据那一个规则生成.o文件。（这有点像一个栈的过程）
- 5、前提:.c文件和.h文件是存在



make具体工作

- 这就是整个make的依赖性，make会一层又一层地去找文件的依赖关系，直到最终编译出第一个目标文件。
- 在找寻的过程中，如果出现错误，比如最后被依赖的文件找不到，那么make就会直接退出，并报错，而对于所定义的命令的错误，或是编译不成功，make根本不理。
- 通过上述分析，我们知道，像clean这种，没有被第一个目标文件直接或间接关联，那么它后面所定义的命令将不会被自动执行，不过，我们可以显示要make执行。即命令——“make clean”，以此来清除所有的目标文件，以便重编译。



在Makefile中使用变量

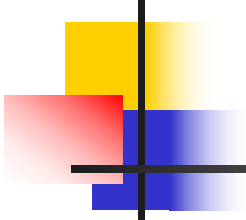
■ makefile里的变量就像一个环境变量，一般使用大写字母。
变量的主要作用如下：

■ 保存文件名

例如：使用一个变量来保存所有的目标文件名，则可以方便地加入新的目标文件而且不易出错。

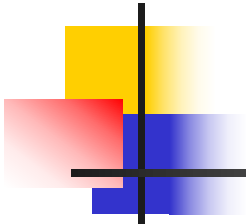
■ 保存可执行命令名

如：编译命令，用一个变量来代替编译器名，那么只需要改变该变量的值。其他所有地方的命令名就都改变了。



■ 保存编译器的参数

很多源代码编译时，gcc需要很长的参数选项，
在很多情况下，所有的编译命令使用一组相同的选项，
如果把这组选项使用一个变量代表，那么可以把这个变量放在所有引用编译器的地方，
当要改变选项的时候，只需改变一次这个变量的内容即可。



1. 变量定义和引用

- 变量（又称做宏定义）一般均由大写字母和数字组成。
- 定义变量的一般格式是：

<变量名>=<字符串>

例如： `OBJECT=x.o y.o z.o`

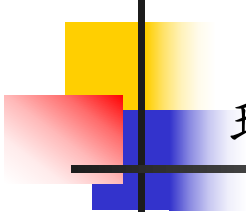
`LIBES=-lm`

- 引用make变量的方式与引用shell变量类似，

把变量用圆括号括起来，并在前面加上“\$”符号。例如：

`$(OBJECT)`

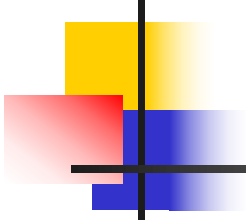
`$(LIBES)`



现在利用变量把前面的makefile文件重写一遍：

```
test: prog.o code.o
    gcc -o test prog.o code.o
prog.o: prog.c prog.h code.h
    gcc -c prog.c -o prog.o
code.o: code.c code.h
    gcc -c code.c -o code.o
clean:
    rm -f *.o
```

```
OBJS=prog.o code.o
CC=gcc
test: $(OBJS)
    $(CC) -o test $(OBJS)
prog.o: prog.c prog.h code.h
    $(CC) -c prog.c -o prog.o
code.o: code.c code.h
    $(CC) -c code.c -o code.o
clean:
    rm -f *.o
```



使用变量

取得变量值的方式: $\$(变量名)$ 或 $\${变量名}$

定义变量

- $=$

- 使用时展开

- $:=$

- 定义时即展开(只能使用已经定义好的变量)

- $+=$

- 变量追加

- $?=$

- 若未定义则定义, 若已定义则不执行此定义

例如:

- `objects = main.o kbd.o command.o display.o \`
`insert.o search.o files.o utils.o`

- 于是, 我们就可以很方便地在我们的makefile中以 “ $\$(objects)$ ”的方式来使用这个变量了

- `objects = program.o foo.o utils.o`

- `program : $(objects)`

- `cc -o program $(objects)`

或者：`cc $^ -o $@`

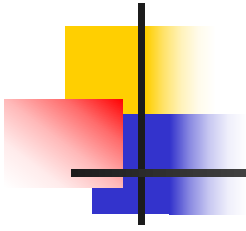
`$^`和`$@`属于自动化变量

①`$@`表示规则中的目标文件集。在模式规则中，如果有多个目标，那么，"`$@`"就是匹配于目标中模式定义的集合。这里是
`program`

②`$^`所有的依赖目标的集合。以空格分隔。如果在依赖目标中有多个重复的，那个这个变量会去除重复的依赖目标，只保留一份。这里是`$(objects)`

- `$(objects) : defs.h`

变量会在使用的地方精确地展开，就像C/C++中的宏一样



OBJS = file1.o file2.o

CC = gcc

CFLAGS = -Wall -O -g

-Wall: 输出所有的警告信息; -O: 在编译时进行优化; -g: 表示编译debug版本。

helloworld : \$(OBJS)

\$(CC) \$(OBJS) -o helloworld

file1.o : file1.c file2.h

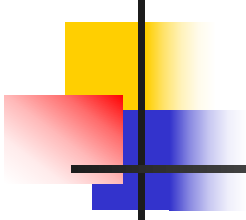
\$(CC) \$(CFLAGS) -c file1.c -o file1.o

file2.o : file2.c file2.h

\$(CC) \$(CFLAGS) -c file2.c -o file2.o

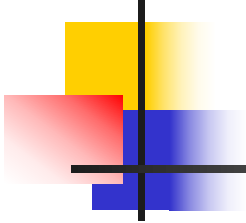
clean:

rm -rf *.o helloworld



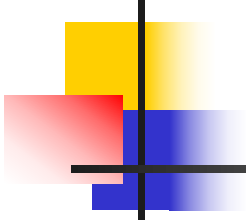
自动推导-隐晦规则？

- GNU的make很强大，它可以自动推导文件以及文件依赖关系后面的命令，于是我们就没必要去在每一个[.o]文件后都写上类似的命令，因为，我们的make会自动识别，并自己推导命令。
- 只要make看到一个[.o]文件，它就会自动的把[.c]文件加在依赖关系中，如果make找到一个whatever.o，那么 whatever.c，就会是whatever.o的依赖文件。并且 gcc -c whatever.c 也会被推导出来，于是，我们的makefile再也不用写得这么复杂。我们新的makefile又出炉了。

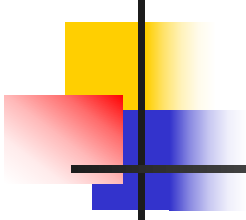


前面的makefile可以这么写：

```
objects = main.o kbd.o command.o display.o \  
insert.o search.o files.o utils.o  
edit : $(objects)  
cc -o edit $(objects)  
main.o : defs.h  
kbd.o : defs.h command.h  
command.o : defs.h command.h  
display.o : defs.h buffer.h  
insert.o : defs.h buffer.h  
search.o : defs.h buffer.h  
files.o : defs.h buffer.h command.h  
utils.o : defs.h  
.PHONY : clean  
clean :  
rm edit $(objects)
```

- 
- Make又是如何实现自动推导的呢？在隐含规则中的命令中，基本上都是使用了一些预先设置的变量。你可以在你的makefile中改变这些变量的值。
 - 比如，编译C程序的隐含规则的命令是：
`$(CC) -c $(CFLAGS) $(CPPFLAGS)`
 - Make默认的编译命令是“cc”，如果你把变量“\$(CC)”重定义成“gcc”，把变量“\$(CFLAGS)”重定义成“-g”，那么，隐含规则中的命令全部会以“gcc -c -\$(CPPFLAGS)”的样子来执行了。

我们可以把隐含规则中使用的变量分成两种：一种是命令相关的，如“CC”；一种是参数相关的，如“CFLAGS”。



■ 下面是C/C++隐含规则中会用到的变量：

■ 关于命令的变量：

AR 函数库打包程序。默认命令是“ar”。

AS 汇编语言编译程序。默认命令是“as”。

CXX C++语言编译程序。默认命令是“g++”。

CC C语言编译程序。默认命令是“cc”。

CPP C程序的预处理器(输出是标准输出设备)。默认命令是\$(CC) -E

■ 关于命令参数的变量。下面的这些变量都是相关上面的命令的参数。
如果没有指明其默认值，那么其默认值都是空。

ARFLAGS 函数库打包程序AR命令的参数。默认值是“rv”。

ASFLAGS 汇编语言编译器参数。（当明显地调用“.s”或“.S”文件时）

CFLAGS C语言编译器参数。

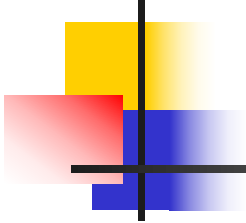
CXXFLAGS C++语言编译器参数。

LDFLAGS 链接器参数。（例如：LDFLAGS += -L ./ -ltest）

CPPFLAGS C预处理器参数。（C和Fortran编译器也会用到）。

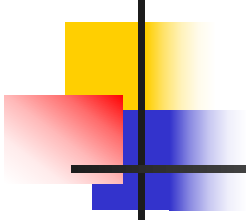
Makefile的文件名

- 默认的情况下，make命令会在当前目录下按顺序找寻文件名为“GNUmakefile”、“makefile”、“Makefile”的文件，找到了就解释这个文件。在这三个文件名中，最好使用“Makefile”这个文件名，因为，这个文件名第一个字符为大写，这样有一种显目的感觉。最好不要用“GNUmakefile”，这个文件是GNU的make识别的。有另外一些make只对全小写的“makefile”文件名敏感，但是基本上来说，大多数的make都支持“makefile”和“Makefile”这两种默认文件名。
- 当然，你可以使用别的文件名来书写Makefile，比如：“Make.Linux”，“Make.Solaris”，“Make.AIX”等，如果要指定特定的Makefile，你可以使用make的“-f”和“--file”参数，如：make -f Make.Linux或make --file Make.AIX。



引用其它的Makefile

- 在Makefile使用include关键字可以把别的Makefile包含进来，这很像C语言的#include，被包含的文件会原模原样的放在当前文件的包含位置。include的语法是：
- `include <filename>`
- filename可以是当前操作系统Shell的文件模式（可以包含路径和通配符）
- 在include前面可以有一些空字符，但是绝不能是[Tab]键开始。include和<filename>可以用一个或多个空格隔开。举个例子，你有这样几个Makefile：a.mk、b.mk、c.mk，还有一个文件叫foo.make，以及一个变量\$(bar)，其包含了e.mk和f.mk，那么，下面的语句：
- `include foo.make *.mk $(bar)`



综合作业和思政
(网络教学平台, 提交截止日期
5.31日, 切勿抄袭!)